UNIVERSITÉ PARIS 8 DES CRÉATIONS

Programmation Objet Avancée

Cours 2 - Modéliser

Ahmed ADJIBADE

ahmed.adjibade@bivariant.com

Année universitaire : 2026-2027

Modéliser

Modéliser, c'est traduire les règles du métier en types Java.

Un bon modèle rend certaines erreurs impossibles, et fait apparaître les autres dès la compilation.

Partie	Question à se poser
Valeurs et entités	Cet objet a-t-il une identité ?
L'égalité	Quand deux objets sont-ils "les mêmes" ?
Les exceptions	Que faire quand une règle est violée ?
Types scellés et motifs	Comment traiter une liste fermée de cas ?
Immutabilité et composition	Comment empêcher les modifications imprévues ?

Nos exemples porteront sur une médiathèque et nos exercices sur notre système de billetterie.

Valeurs et entités

Deux sortes d'objets : valeurs et entités

- Échange d'un billet de 10€ contre celui de votre voisin ⇒ Seule la valeur compte
- Remplacez votre voisin par un inconnu qui porte le même nom ⇒ Ce n'est plus la même personne !!

Le billet est une valeur … la personne une entité



Je retiens : Avant d'écrire une classe, il faut se demander si deux objets de même contenu sont interchangeables. Si oui, c'est une valeur.

Écrire une valeur à la main

- Une durée d'emprunt est une valeur ⇒ deux durées de 21 jours doivent être égales
- Une classe "classique" doit alors tout écrire à la main ⇒ attribut, constructeur, accesseur, *equals*, *hashCode*, *toString*
- Ajouter une donnée ⇒ Cinq endroits à modifier, et un oubli dans *equals* ne se voit pas …

Il faut environ 15 lignes pour dire une seule chose … une durée c'est un nombre de jours !



Écrire une valeur à la main

Fichier : Duree.java

```java
public final class Duree {

    private final int jours;

    public Duree(int jours) {

        this.jours = jours;

    }

    public int jours() { return jours; }

    @Override

    public boolean equals(Object autre) {

        return autre instanceof Duree duree && jours == duree.jours;

    }

    @Override

    public int hashCode() { return Integer.hashCode(jours); }

    @Override

    public String toString() { return "Duree[jours=" + jours + "]"; }

}
```

Le record

- Un record est une classe dont le seul rôle est de **transporter des données**.
- Ces données, ses **composants**, se déclarent entre parenthèses ⇒ ici, un seul composant : `int jours`
- Le compilateur en déduit tout le reste ⇒ le code de la slide précédente est générée automatiquement

Le record

Fichier : Duree.java

```
public record Duree(int jours) {

}
```

Ces deux lignes remplacent les 15 lignes précédentes. Les records existent depuis Java 16.

Je retiens : *record* se lit “ces données, et rien d’autre”. Un objet entièrement décrit par ses données est un bon candidat.

Le compilateur, que génère-t-il vraiment ?

Java génère …	Pour record Duree(int jours)
Un attribut `private final` par composant	`private final int jours;`
Le constructeur **canonique** : un paramètre par composant, dans le même ordre	`new Duree(21)`
Un accesseur par composant, qui porte son nom (sans get)	jours()
`equals`, qui compare **tous** les composants	`new Duree(21).equals(new Duree(21))` vaut `true`
`hashCode`, calculé à partir de tous les composants	Même durée ⇒ même `hashCode`
`toString`, avec le nom et la valeur des composants	`Duree[jours=21]`

Le compilateur, que génère-t-il vraiment ?

La classe générée est `final` et hérite de `java.lang.Record` ⇒ ON NE PEUT PAS L'ÉTENDRE.

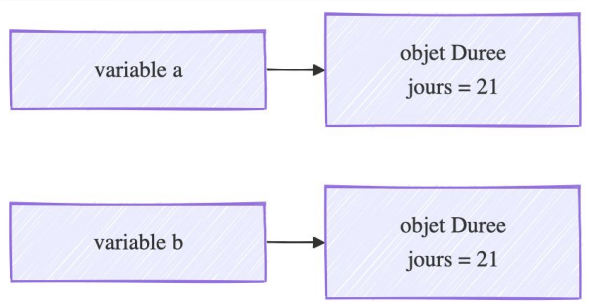
Je retiens : Tout est déduit des composants. Si on ajoute un composant, `equals`, `hashCode` et `toString` le prennent en compte tout seuls !

Utiliser un record : equals ou == ?

- Une variable ne contient pas l'objet, mais une **référence** vers lui, comme une flèche ⇒ deux **new**, deux objets et deux flèches
- `a == b` compare les flèches ⇒ **faux** : elles désignent deux objets différents
- `a.equals(b)` compare le contenu des objets ⇒ **vrai** : même durée

```
Duree a = new Duree(21);
Duree b = new Duree(21);

System.out.println(a.jours());      // 21
System.out.println(a.equals(b));    // true : même contenu
System.out.println(a == b);         // false : deux objets distincts
System.out.println(a);              // Duree[jours=21]
```



Je retiens : Pour comparer des valeurs, on utilise **toujours** `equals`. `==` répond seulement à la question "est-ce le même objet en mémoire ?"

Un record est immuable

- Ses attributs sont final ⇒ Ils sont fixés par le constructeur, puis ne changent plus
- Il n'a pas de setter ⇒ aucune méthode ne peut modifier une valeur existante
- C'est voulu … une durée de 21 jours qui deviendrait 28 jours ne serait plus la même valeur ⇒ on crée une autre durée.

Vous connaissez déjà une valeur immuable : `String`

```
String titre = "dune";

String majuscules = titre.toUpperCase();

System.out.println(titre);        // dune : inchangé

System.out.println(majuscules);   // DUNE : un nouvel objet
```

Je retiens : Immuable signifie que l'état est fixé à la construction et ne change plus jamais. Pour "modifier" une valeur, on en crée une autre.

Valider : le constructeur compact

- Le constructeur généré accepte tout ⇒ `new Duree(-3)` créerait une durée absurde
- Le constructeur est le seul point d'entrée ⇒ c'est là qu'on vérifie, une fois pour toutes
- Le constructeur compact ne contient que les vérifications ⇒ Java ajoute ensuite les affectations

Fichier : Duree.java

```
public record Duree(int jours) {

    public Duree {

        if (jours < 0) {

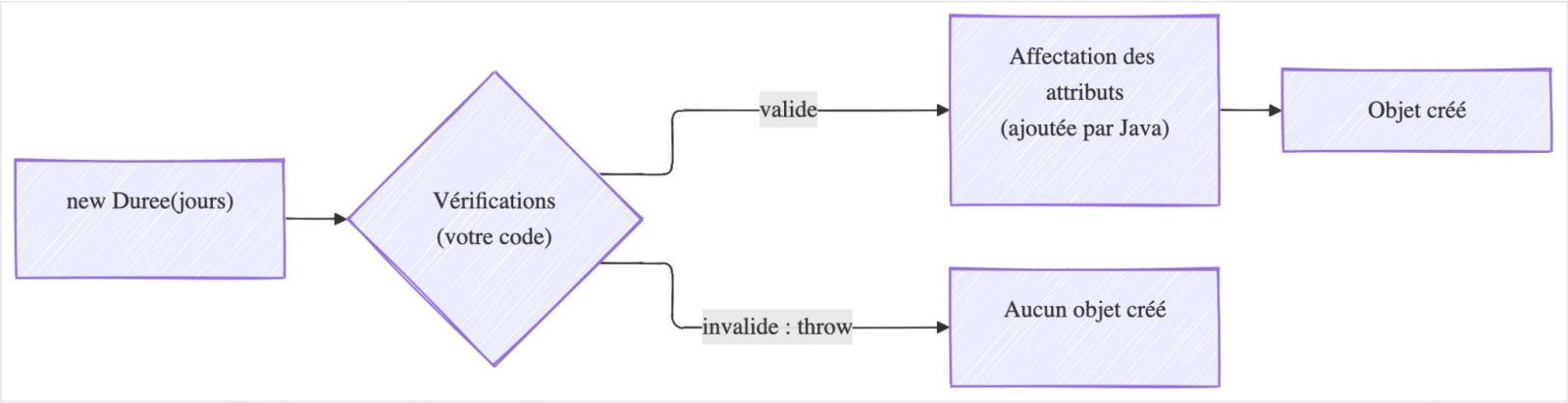
            throw new IllegalArgumentException("Durée négative : " + jours);

        }

    }

}
```

Valider : le constructeur compact



Je retiens : si une Duree existe, elle est valide. C'est garanti à un seul endroit du code.

Attention : `this.jours = jours;` est refusé dans un constructeur compact (`cannot assign a value to final variable jours`), car l'affectation est le travail de Java.

Calculer avec des valeurs : produire, pas modifier

- `plus` ne touche pas à **`this`** ⇒ elle construit et renvoie une nouvelle `Duree`, validée elle aussi par le constructeur compact
- `UNE_SEMAINE` est une constante **`static`** ⇒ partagée par tout le programme, sans risque puisqu'elle ne peut pas changer

```
public record Duree(int jours) {
    public static final Duree UNE_SEMAINE = new Duree(7);
    public Duree {
        if (jours < 0) {
            throw new IllegalArgumentException("Durée négative : " + jours);
        }
    }
    public Duree plus(Duree autre) {
        return new Duree(jours + autre.jours);
    }
}
```

Calculer avec des valeurs : produire, pas modifier

```java
Duree emprunt = new Duree(21);
Duree prolonge = emprunt.plus(Duree.UNE_SEMAINE);

System.out.println(emprunt);    // Duree[jours=21] : inchangée
System.out.println(prolonge);   // Duree[jours=28]
```

Je retiens : Une méthode de record renvoie un nouvel objet. Elle ne modifie jamais **`this`**.

Ce qu’un record ne peut pas faire

Interdit	Parce que
Ajouter un attribut d’instance	Tout l’état est dans les composants (`field declaration must be static`)
Modifier un composant	Un record est immuable
Hériter d’une classe (`extends`)	Il hérite déjà de `java.lang.Record`
Être étendu	Il est `final` … une sous-classe pourrait ajouter un état caché

Un record peut en revanche implémenter des interfaces, et avoir des méthodes, des constantes `static` et un constructeur compact.

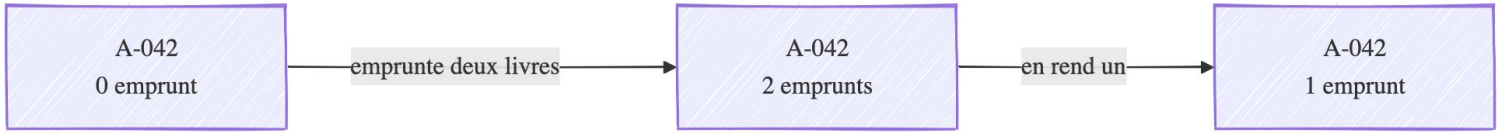
Je retiens : Le record est fait pour une valeur. Pour une entité, dont l’état évolue, on écrit une classe.

L’égalité

Une entité : une identité qui traverse le temps

- Une entité a un **identifiant** qui ne change jamais ⇒ le numéro de carte d'un adhérent
- Son **état**, lui, évolue ⇒ elle emprunte, elle rend, elle déménage
- Deux objet représentent la même entité s'ils ont le même identifiant ⇒ même si leur état diffère

Alice qui déménage reste Alice : c'est son numéro de carte qui le dit, pas son adresse.



Je retiens : Une valeur est définie par son contenu, une entité par son identifiant.

Écrire une entité

- L'identifiant est `final` ⇒ il ne peut plus changer après la construction
- L'état (`empruntsEnCours`) n'est pas `final` ⇒ il évolue avec `emprunter()`
- `Objects.requireNonNull` refuse un identifiant `null` ⇒ l'erreur est signalée tout de suite, et non plus tard, loin de sa cause

Écrire une entité

Fichier : Adherent.java

```java
import java.util.Objects;

public final class Adherent {

    private final String numeroCarte;

    private final String nom;

    private int empruntsEnCours;

    public Adherent(String numeroCarte, String nom) {

        this.numeroCarte = Objects.requireNonNull(numeroCarte, "numeroCarte");

        this.nom = nom;

    }

    public String numeroCarte() { return numeroCarte; }

    public String nom() { return nom; }

    public void emprunter() {

        empruntsEnCours++;

    }

}
```

Je retiens : On écrit une classe et non un record, car l'état change et l'égalité ne porte pas sur tout l'état.

Quid de l'égalité par défaut provenant d'Object ?

- Toute classe hérite `d'Object` ⇒ donc de sa méthode equals
- L'`equals` d'`Object` compare les références ⇒ il fait exactement la même chose que `==`
- Or un programme crée souvent deux objets pour la même entité ⇒ une fiche lue deux fois dans un fichier, une autre saisie dans un formulaire…

```
Adherent a = new Adherent("A-042", "Alice Martin");
Adherent b = new Adherent("A-042", "Alice Martin");
System.out.println(a.equals(b));   // false : la même adhérente, deux objets
```

Je retiens : Redéfinir `equals`, c'est dire à Java ce que « le même » signifie pour cette classe.

Redéfinir equals

- Le paramètre est de type `Object` ⇒ on peut recevoir n'importe quel objet, et même `null`
- autre instanceof Adherent adherent ⇒ vérifie le type, écarte null et déclare la variable adherent, déjà convertie
- On ne compare que les identifiants ⇒ deux Adherent de même numéro représentent la même personne

```
@Override
 public boolean equals(Object autre) {
     return autre instanceof Adherent adherent
             && numeroCarte.equals(adherent.numeroCarte);
 }
```

Redéfinir equals

Avant Java 16, la méthode s'écrivait:

```
@Override
    public boolean equals(Object autre) {
        if (!(autre instanceof Adherent)) {
            return false;
        }
        Adherent adherent = (Adherent) autre;
        return numeroCarte.equals(adherent.numeroCarte);
    }
```

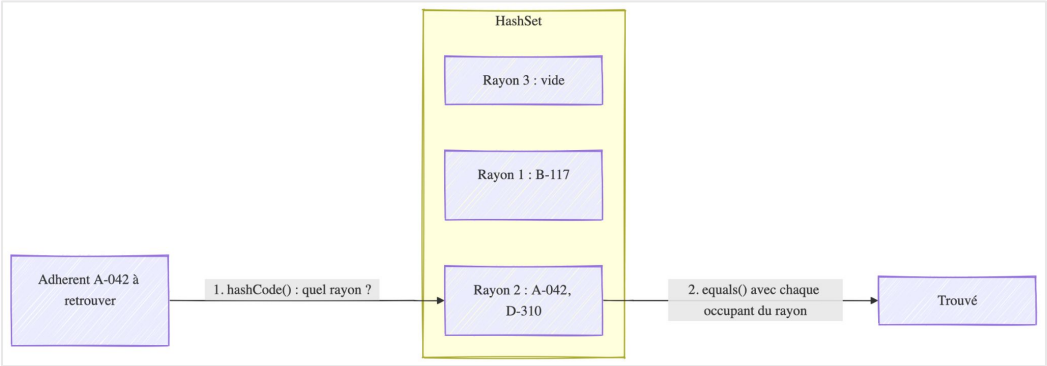
Je retiens :

- La signature `equals(Object)` est imposée par `Object`.
- On peut lire `adherent.numeroCarte` bien qu'il soit privé : **`private`** protège la classe, pas chaque objet.

hashCode : le numéro de rayon

Dans une médiathèque, on ne cherche pas un livre en parcourant tous les rayons. La cote indique le rayon, puis on compare seulement les livres de ce rayon.

- Un `HashSet` qui comparerait avec tous ses éléments serait trop lent ⇒ un million d'adhérents, un million d'appels à `equals`
- `hashCode()` calcule un numéro de rayon ⇒ on ne cherche que dans ce rayon
- `equals()` compare ensuite avec les quelques objets du rayon



Je retiens : hashCode dit où chercher, equals dit si c'est bien lui.

Le contrat entre equals et hashCode

- Deux objets égaux doivent avoir le même `hashCode` ⇒ sinon, ils sont rangés dans des rayons différents, et le second n'est jamais trouvé
- L'inverse n'est pas exigé ⇒ plusieurs objets différents peuvent partager un rayon
- Donc `equals` et `hashCode` se redéfinissent ensemble, sur les mêmes attributs

```
@Override
public int hashCode() {
    return numeroCarte.hashCode();
}
```

Le contrat entre equals et hashCode

Les autres règles de equals	Concrètement
Réflexivité	`a.equals(a)` est vrai
Symétrie	`a.equals(b)` vaut `b.equals(a)`
Transitivité	si `a.equals(b)` et `b.equals(c)`, alors `a.equals(c)`
`null`	`a.equals(null)` est faux

Je retiens : equals et hashCode se redéfinissent toujours ensemble, sur l'identifiant.

Ce qui casse (1) : equals sans hashCode

- Pour `equals`, les deux objets sont égaux
- Sans redéfinition, `hashCode` dépend de l'objet en mémoire ⇒ deux rayons différents
- Le `HashSet` ne les compare donc jamais ⇒ il garde les deux

```
Set<Adherent> inscrits = new HashSet<>();
inscrits.add(new Adherent("A-042", "Alice Martin"));
inscrits.add(new Adherent("A-042", "Alice Martin"));

System.out.println(inscrits.size());   // 1 avec hashCode, 2 sans
```

Je retiens : `equals` sans `hashCode` produit des doublons dans un `HashSet`.

Ce qui casse (2) : un hashCode qui change

Par hypothèse : `hashCode` est écrit `Objects.hash(numeroCarte, empruntsEnCours)`.

- Alice est rangée dans le rayon calculé avec 0 emprunt
- Après un emprunt, son `hashCode` change ⇒ `contains` cherche dans un autre rayon
- Alice est dans l'ensemble, mais introuvable

```
Adherent alice = new Adherent("A-042", "Alice Martin");
Set<Adherent> inscrits = new HashSet<>();
inscrits.add(alice);
alice.emprunter();
System.out.println(inscrits.contains(alice));   // false : Alice a changé de rayon
```

Je retiens : `equals` et `hashCode` reposent seulement sur ce qui ne change jamais, c'est-à-dire l'identifiant.

Le piège de la surcharge

- `equals(Adherent)` ne remplace pas `equals(Object)` ⇒ elle s'ajoute à côté : c'est une surcharge, pas une redéfinition
- `HashSet` et `assertEquals` appellent `equals(Object)` ⇒ ils utilisent toujours celui d'`Object`

```
public boolean equals(Adherent autre) {
    return autre != null && numeroCarte.equals(autre.numeroCarte);
}
```

Avec `@Override`, le compilateur refuse ce code

```
error: method does not override or implement a method from a supertype
```

Je retiens : `@Override` sur chaque redéfinition. L'annotation ne coûte rien et transforme une erreur silencieuse en erreur de compilation.

Les exceptions

Qu’est-ce qu’une exception ?

- Une méthode ne peut pas faire ce qu’on lui demande ⇒ au lieu de renvoyer un résultat, elle **lève** une exception
- Une exception est un objet ⇒ son type dit ce qui s’est passé, son message donne les détails
- Elle interrompt le programme là où elle est levée ⇒ les lignes suivantes ne s’exécutent pas

Pourquoi ne pas renvoyer une valeur spéciale, comme -1 ou null ? Parce que l’appelant peut l’oublier sans s’en rendre compte. Une exception ne passe pas inaperçue : si personne ne la traite, elle arrête le programme.

```
throw new IllegalArgumentException("Durée négative : " + jours);
```

Je retiens : `new` crée l’objet exception, `throw` le lance.

Lever et attraper

Par hypothèse : `guichet.prolonger(duree, jours)` prolonge un emprunt et appelle `new Duree(jours)`

- `try` entoure le code qui peut échouer
- Dès qu'une exception est levée, le reste du `try` est sauté ⇒ « Nouvelle durée » ne s'affiche pas
- Le `catch` du bon type s'exécute, puis le programme reprend après le bloc

Lever et attraper

```
Guichet guichet = new Guichet();
try {
    Duree duree = guichet.prolonger(new Duree(21), -3);
    System.out.println("Nouvelle durée : " + duree);
} catch (IllegalArgumentException e) {
    System.out.println("Refusé : " + e.getMessage());
}
System.out.println("Le programme continue");
```

Refusé : Durée négative : -3

Le programme continue

Je retiens : Attraper une exception, c'est décider de ce qu'on fait quand l'opération échoue.

La remontée de la pile d'appels

Fichier Guichet.java

```
public class Guichet {

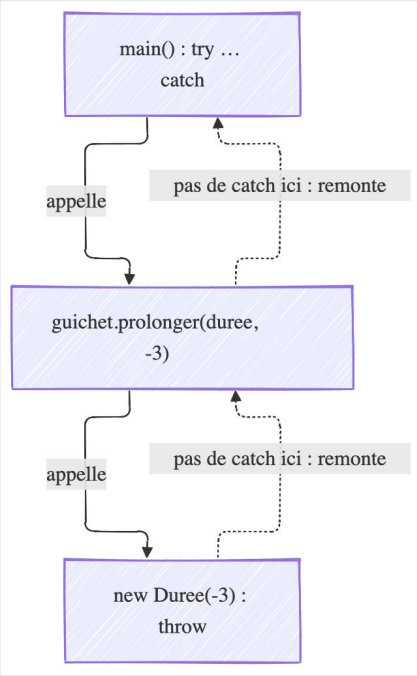
    public Duree prolonger(Duree actuelle, int joursEnPlus) {

        return actuelle.plus(new Duree(joursEnPlus));

    }

}
```

- main appelle prolonger, qui appelle le constructeur de Duree ⇒ c'est la pile d'appels
- L'exception naît dans le constructeur ⇒ pas de catch ici : elle remonte dans prolonger
- Pas de catch dans prolonger non plus ⇒ elle remonte dans main, où le catch la traite



Je retiens : Une exception remonte de méthode en méthode jusqu'au premier catch de son type. Chaque méthode traversée s'arrête net.

Lire une stack trace

Sans `try … catch`, personne n'attrape l'exception et le programme s'arrête :

Fichier Main.java

```
public class Main {
    public static void main(String[] args) {
        Guichet guichet = new Guichet();
        Duree duree = guichet.prolonger(new Duree(21), -3);
        System.out.println(duree);
    }
}
```

```
Exception in thread "main" java.lang.IllegalArgumentException: Durée négative : -3
      at Duree.<init>(Duree.java:7)
      at Guichet.prolonger(Guichet.java:3)
      at Main.main(Main.java:4)
```

Lire une stack trace

```
Exception in thread "main" java.lang.IllegalArgumentException: Durée négative : -3
      at Duree.<init>(Duree.java:7)
      at Guichet.prolonger(Guichet.java:3)
      at Main.main(Main.java:4)
```

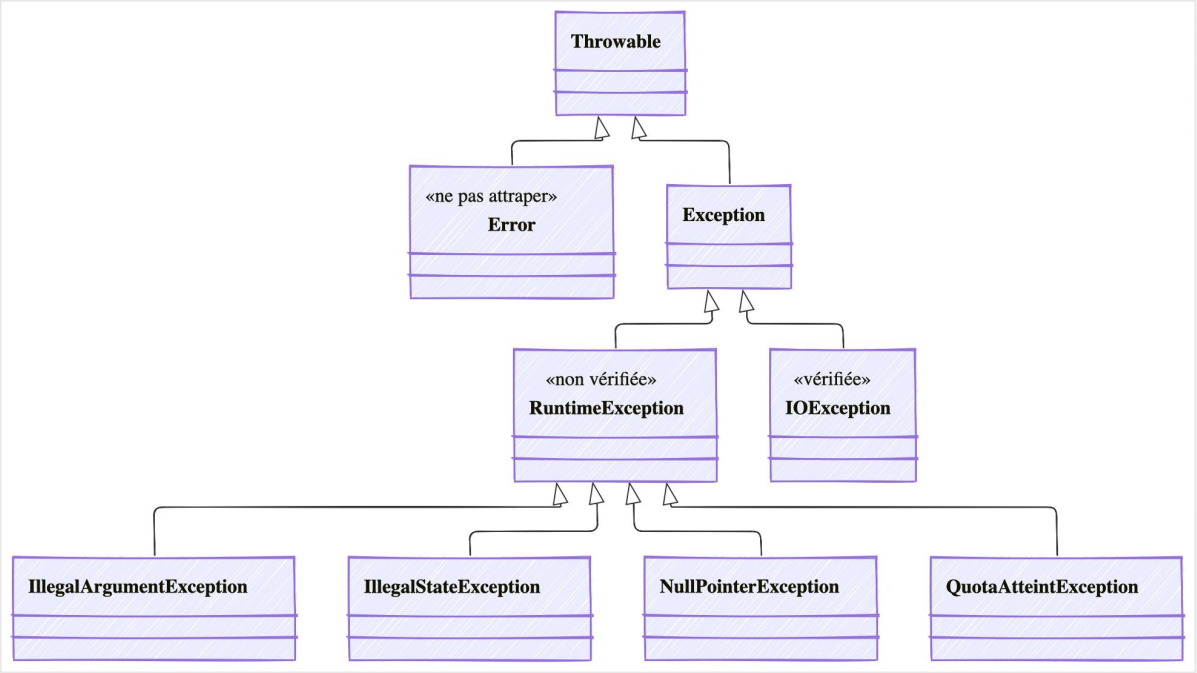
- Ligne 1 : le type de l'exception et son message ⇒ ce qui s'est passé
- Ligne 2 : l'endroit où elle est née ⇒ <init> désigne un constructeur, et Duree.java:7 la ligne du throw
- Lignes suivantes : le chemin des appels, du plus récent au plus ancien ⇒ cherchez la première ligne qui concerne votre code

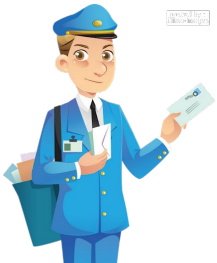
Je retiens : Une stack trace se lit de haut en bas. La réponse est presque toujours dans les trois premières lignes.

Deux familles : vérifiées et non vérifiées

Une exception vérifiée ressemble à une **lettre recommandée** : impossible de faire comme si on ne l'avait pas reçue, le compilateur exige une signature.

Une exception non vérifiée ressemble à une lettre simple : on peut la traiter, mais rien ne l'impose.





Deux familles : vérifiées et non vérifiées

- Non vérifiée : `RuntimeException` et ses sous-classes ⇒ une erreur de programmation ou une règle violée. Le compilateur n'impose rien
- Vérifiée : toutes les autres `Exception` ⇒ un aléa extérieur dont on peut se remettre (fichier absent, réseau coupé). Le compilateur impose une réponse
- `Error` : un problème grave de la machine virtuelle, comme une mémoire épuisée ⇒ on ne l'attrape pas

Je retiens : C'est la position dans la hiérarchie qui décide si une exception est vérifiée.

Une exception vérifiée exige une réponse

```
String texte = Files.readString(Path.of("catalogue.csv"));
```

```
error: unreported exception IOException; must be caught or declared to be
thrown
```

- Le fichier peut ne pas exister ⇒ le compilateur oblige à le prévoir
- Première réponse : l'attraper avec `try … catch`
- Seconde réponse : la transmettre à l'appelant, en la déclarant avec throws

Une exception vérifiée exige une réponse

Fichier Catalogue.java

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class Catalogue {
    public String lire() throws IOException {
        return Files.readString(Path.of("catalogue.csv"));
    }
}
```

Je retiens : `throws` fait passer la lettre recommandée à l'appelant, qui devra à son tour l'attraper ou la transmettre. Nous travaillerons sur les fichiers plus tard.

Choisir son exception

Situation	Exception
Un argument a une valeur interdite	`IllegalArgumentException`
Un argument obligatoire vaut `null`	`NullPointerException`, avec `Objects.requireNonNull`
L'opération est interdite dans l'état actuel de l'objet	`IllegalStateException`
Une règle du métier est violée, et l'appelant a besoin de détails	Une exception métier, qui hérite de `RuntimeException`
Un aléa extérieur (fichier, réseau)	Une exception vérifiée : `IOException…`

Je retiens : On réutilise d'abord les exceptions standard. On crée la sienne quand son nom ou ses données apportent quelque chose à l'appelant.

Créer une exception métier

Par hypothèse : un adhérent ne peut pas avoir plus de 5 emprunts en cours.

- Un nom précis ⇒ l'appelant peut attraper ce cas, et seulement lui
- Des données ⇒ l'appelant les lit sans avoir à analyser le message
- Elle hérite de `RuntimeException` ⇒ elle est non vérifiée, comme le veut l'usage pour une règle du métier

Fichier QuotaAtteintException.java

```java
public class QuotaAtteintException extends RuntimeException {
    private final int quota;

    public QuotaAtteintException(String numeroCarte, int quota) {
        super("Adhérent " + numeroCarte + " : quota de " + quota + " emprunts atteint");
        this.quota = quota;
    }

    public int quota() { return quota; }
}
```

Je retiens : `super(message)` transmet le message à la classe parente. On le récupère ensuite avec `getMessage()`

Lever et exploiter une exception métier

Fichier : Adherent.java (suite) : emprunter vérifie désormais le quota.

```
public static final int QUOTA = 5;

    public void emprunter() {
        if (empruntsEnCours == QUOTA) {
            throw new QuotaAtteintException(numeroCarte, QUOTA);
        }
        empruntsEnCours++;
    }
```

Lever et exploiter une exception métier

```
try {
    alice.emprunter();
    System.out.println("Emprunt enregistré");
} catch (QuotaAtteintException e) {
    System.out.println("Quota de " + e.quota() + " atteint : rendez d'abord un
document");
}
```

- On attrape le type précis ⇒ les autres erreurs continuent de remonter
- `e.quota()` donne la donnée utile ⇒ l'appelant construit son propre message
- `e.getMessage()` sert à journaliser ⇒ « Adhérent A-042 : quota de 5 emprunts atteint »

Je retiens : L'exception métier transporte tout ce dont l'appelant a besoin pour réagir.

Valider avant de modifier

À ne pas faire :

```
public void emprunter() {
    empruntsEnCours++;
    if (empruntsEnCours > QUOTA) {
        throw new QuotaAtteintException(numeroCarte, QUOTA);
    }
}
```

- L'emprunt est compté, puis refusé ⇒ après le refus, l'adhérent compte 6 emprunts
- L'opération a échoué, mais elle a modifié l'objet ⇒ l'objet est corrompu

Je retiens : D'abord toutes les vérifications, ensuite les modifications. Une opération refusée laisse l'objet dans son état d'origine.

Valider avant de modifier

Attention : un `catch` vide fait disparaître l'erreur sans laisser de trace.

```
try {
    alice.emprunter();
} catch (Exception e) {
}
```

N'attrapez que ce que vous savez traiter, avec le type le plus précis possible.

Types scellés et motifs

Le problème : une hiérarchie ouverte

- Au cours 1, `Document` était une classe abstraite ⇒ n'importe qui, n'importe où, pouvait en créer une sous-classe
- Le compilateur ne connaît donc jamais la liste complète des documents ⇒ impossible de vérifier qu'un traitement les couvre tous
- Une branche « sinon » devient obligatoire ⇒ un nouveau type y tombe en silence

```
static Duree dureeEmprunt(Document document) {
    if (document instanceof Livre) {
        return new Duree(21);
    } else if (document instanceof Dvd) {
        return Duree.UNE_SEMAINE;
    } else {
        return Duree.UNE_SEMAINE;   // les revues… et tout type ajouté plus tard
    }
}
```

Je retiens : Avec une hiérarchie ouverte, oublier un cas ne provoque aucune erreur. On le découvre en production.

Sealed : fermer la liste

Dans une soirée sur invitation, le videur a la liste complète.

À l'appel, il sait dire qui manque. Sans liste, n'importe qui peut entrer, et personne ne peut affirmer que tout le monde est là.

Ici, le **type scellé** c'est la liste et le **compilateur** c'est le videur.

- `sealed` ⇒ la hiérarchie est fermée
- `permits` ⇒ la liste exacte des sous-types autorisés
- Toute autre classe qui tente d'implémenter `Document` ⇒ erreur de compilation



Sealed : fermer la liste

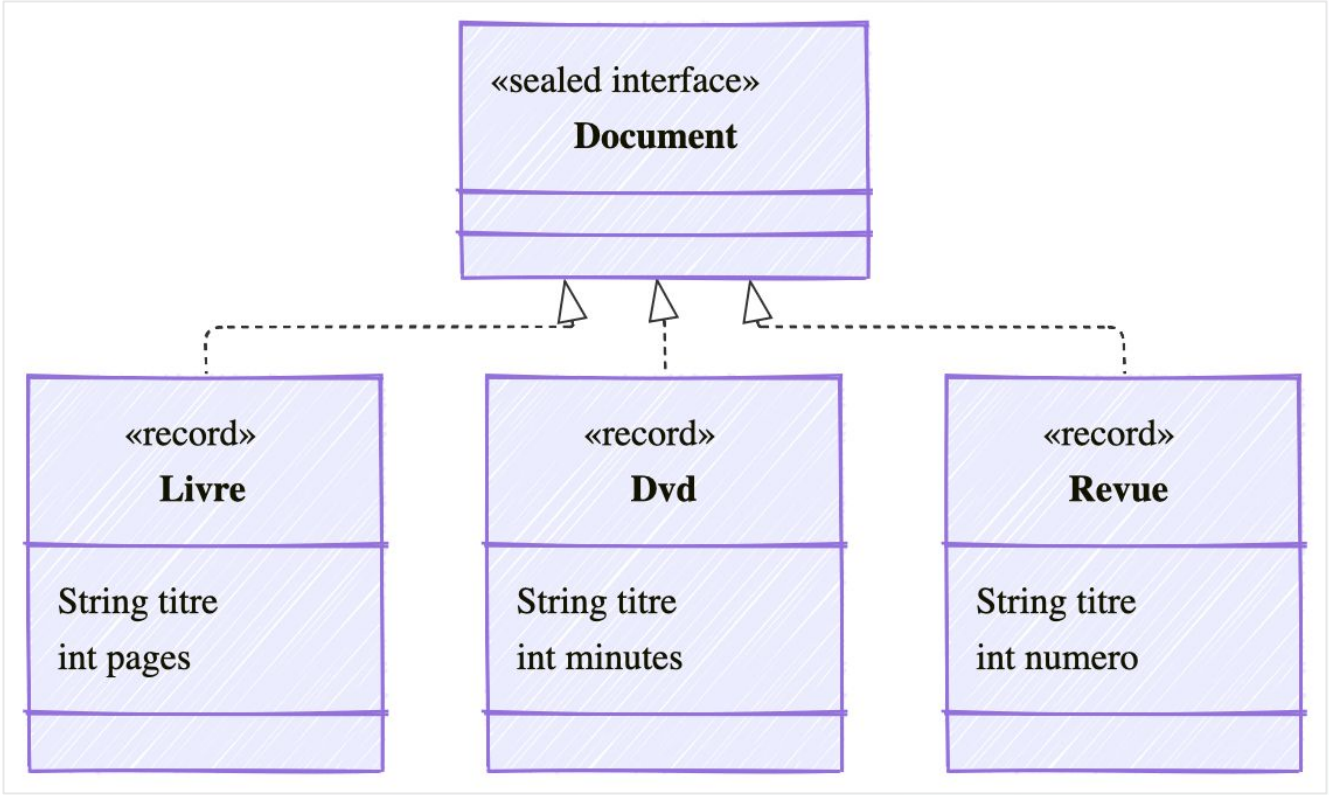
Fichiers : Document.java, Livre.java, Dvd.java et Revue.java

```java
public sealed interface Document permits Livre, Dvd, Revue {}
public record Livre(String titre, int pages) implements Document { }
public record Dvd(String titre, int minutes) implements Document { }
public record Revue(String titre, int numero) implements Document { }
```

```
error: class is not allowed to extend sealed class: Document (as it is not listed in its 'permits' clause)
```

Je retiens : Un type scellé, c'est une liste de cas fermée, connue du compilateur.

Les règles des types scellés



Les règles des types scellés

- Chaque sous-type autorisé est `final`, `sealed` ou `non-sealed` ⇒ un record est implicitement final, il convient donc
- `permits` est obligatoire quand les sous-types sont dans d'autres fichiers ⇒ ils doivent être dans le même package
- Sealed et records vont ensemble ⇒ une liste fermée de cas, chacun avec ses propres données

Par hypothèse : si les cas ne portaient aucune donnée propre, un `enum` suffirait (`enum Support { LIVRE, DVD, REVUE }`). Un type scellé se justifie quand chaque cas a ses propres données : des pages, des minutes, un numéro.

Le pattern matching : tester et extraire en un geste

- Un motif (pattern) décrit une forme ⇒ « un `Livre` », ou « un `Livre`, dont je veux le titre et les pages »
- Si l'objet a cette forme ⇒ le test réussit, et ses morceaux sont rangés dans des variables
- Sinon ⇒ le test échoue, et aucune variable n'est créée

```
// 1. Avant Java 16 : tester, puis convertir à la main
if (document instanceof Livre) {
    Livre livre = (Livre) document;
    System.out.println(livre.pages());
}
```

Le pattern matching : tester et extraire en un geste

```java
// 2. Motif de type : tester et nommer
if (document instanceof Livre livre) {
    System.out.println(livre.pages());
}
// 3. Motif de record : tester et extraire les composants
if (document instanceof Livre(String titre, int pages)) {
    System.out.println(titre + " : " + pages);
}
```

Je retiens : Un motif de type teste et nomme. Un motif de record teste et extrait les composants.

Switch sur le type

- Ce `switch` est une expression ⇒ il produit une valeur, ici renvoyée par `return`
- La flèche -> ⇒ un seul cas s'exécute : pas de `break`, et aucun risque de « tomber » dans le cas suivant
- Chaque `case` est un motif ⇒ le premier qui correspond est choisi
- Pas de `default` ⇒ les trois cas couvrent tous les documents, et le compilateur le sait grâce à sealed

Switch sur le type

Fichier Catalogue.java

```
public final class Catalogue {
    private Catalogue() { }
    public static String description(Document document) {
        return switch (document) {
            case Livre livre -> livre.titre() + ", " + livre.pages() + " pages";
            case Dvd dvd -> dvd.titre() + ", " + dvd.minutes() + " min";
            case Revue revue -> revue.titre() + ", n° " + revue.numero();
        };
    }
}
```

`description(new Livre("Dune", 412))` renvoie `"Dune, 412 pages"`.

Je retiens : switch sur un type scellé : un cas par sous-type, sans default.

Attention : si document vaut null, le switch lève une NullPointerException

Motifs de record dans un switch

- `case Livre(String titre, int pages)` ⇒ teste le type et extrait les composants, en une fois
- Les noms des variables sont libres ⇒ l'ordre et les types sont ceux du record
- `var` est accepté ⇒ `case Livre(var titre, var pages)`

Fichier : Catalogue.java, autre écriture

```
return switch (document) {
        case Livre(String titre, int pages) -> titre + ", " + pages + " pages";
        case Dvd(String titre, int minutes) -> titre + ", " + minutes + " min";
        case Revue(String titre, int numero) -> titre + ", n° " + numero;
    };
```

Je retiens : Avec les motifs de record, on lit directement les données, sans appeler les accesseurs.

Ignorer avec _, affiner avec when

- `_` ⇒ un composant dont on n'a pas besoin
- `case Livre _` ⇒ teste le type seul, sans rien extraire
- `when` ⇒ ajoute une condition au motif, appelée **garde**
- Les cas sont essayés dans l'ordre ⇒ on les écrit du plus précis au plus général

Ignorer avec _, affiner avec when

Fichier : Regles.java

```
public final class Regles {
    private Regles() { }
    public static Duree dureeEmprunt(Document document) {
        return switch (document) {
            case Livre(_, int pages) when pages > 500 -> new Duree(28);
            case Livre _ -> new Duree(21);
            case Dvd _ -> Duree.UNE_SEMAINE;
            case Revue _ -> Duree.UNE_SEMAINE;
        };
    }
}
```

Je retiens : Un gros livre (plus de 500 pages) est prêté 28 jours, les autres livres 21 jours, les DVD et les revues 7 jours.

Attention : dans l'ordre inverse, `case Livre _` capterait tous les livres, et le compilateur refuse le code : `this case label is dominated by a preceding case label`.

L’exhaustivité : le compilateur vérifie

Par hypothèse : la médiathèque prête désormais des partitions.

```
public sealed interface Document permits Livre, Dvd, Revue, Partition { }
public record Partition(String titre, String compositeur) implements Document { }
```

Chaque switch sur Document devient alors une erreur de compilation :

```
Regles.java:6: error: the switch expression does not cover all possible input values
```

L’exhaustivité : le compilateur vérifie

- Le compilateur désigne chaque `switch` à compléter ⇒ ajouter un cas ne peut plus faire oublier un traitement
- Il peut les signaler en plusieurs fois ⇒ corrigez, puis relancez
- Sans `sealed`, n’importe qui pourrait ajouter un document ⇒ le compilateur exigerait un `default`

Attention : un `default` désactive cette vérification. Avec `default -> Duree.UNE_SEMAINE;` tout compile, et une partition est prêtée 7 jours sans que personne l’ait décidé. D’où la règle : pas de default sur un type scellé.

Immutabilité et composition

Pourquoi des objets immuables ?

- Un objet immuable est vérifié une seule fois, dans son constructeur ⇒ il reste valide pour toujours
- Personne ne peut le modifier dans votre dos ⇒ on peut le partager sans crainte
- C'est une clé sûre dans un `HashSet` ⇒ son `hashCode` ne change jamais
- Plusieurs threads peuvent le lire en même temps ⇒ nous le verrons dans les cours à venir

Outil	Effet
Attributs final	Les attributs ne changent plus après le constructeur
Un argument obligatoire vaut `null`	Aucune porte d'entrée pour modifier l'objet
Méthodes qui renvoient un nouvel objet	Duree.plus()
Classe final, ou record	Aucune sous-classe ne peut ajouter d'état modifiable

Je retiens : Un objet qui ne change pas est un objet sur lequel on peut **compter**.

La fuite par une liste

Par hypothèse : un emprunt regroupe un adhérent et les documents qu'il emprunte.

- Le record copie la référence de la liste, pas la liste ⇒ l'appelant et l'emprunt partagent le même objet
- `final` protège la référence, pas le contenu ⇒ on ne peut pas changer de liste, mais on peut changer la liste

Fichier Emprunt.java

```java
import java.util.List;

public record Emprunt(Adherent adherent, List<Document> documents) {

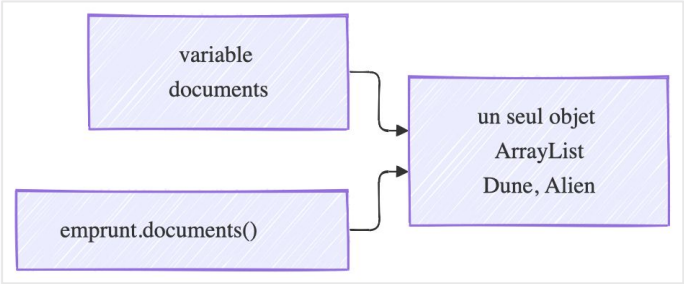
}
```

La fuite par une liste

```java
Adherent alice = new Adherent("A-042", "Alice Martin");
List<Document> documents = new ArrayList<>();
documents.add(new Livre("Dune", 412));
Emprunt emprunt = new Emprunt(alice, documents);

documents.add(new Dvd("Alien", 117));
System.out.println(emprunt.documents().size());   // 2 : l'emprunt a changé dans votre dos

emprunt.documents().clear();
System.out.println(emprunt.documents().size());   // 0
```



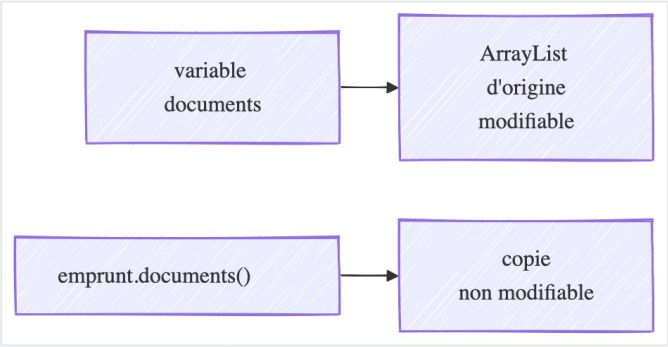
Je retiens : Deux références vers une même liste, et chacune peut la modifier.

La copie défensive

- Le constructeur compact remplace le paramètre par une copie ⇒ c'est la copie qui devient le composant
- La copie est non modifiable ⇒ ni l'appelant ni documents() ne peuvent plus changer l'emprunt
- List.copyOf refuse aussi les éléments null ⇒ une validation de plus, gratuite

Fichier Emprunt.java

```
import java.util.List;
import java.util.Objects;
public record Emprunt(Adherent adherent, List<Document> documents) {
    public Emprunt {
        Objects.requireNonNull(adherent, "adherent");
        documents = List.copyOf(documents);
    }
}
```



Je retiens : Un objet qui reçoit une liste en garde sa propre copie.

Miroir ou photo : choisir sa copie

`Collections.unmodifiableList` est un miroir : on ne peut pas toucher le reflet, mais il change dès que l'original bouge. `List.copyOf` est une photo : elle reste figée, quoi qu'il arrive à l'original.

Écriture	Copie ?	Modifiable par documents() ?
Rien	Non	Oui
`new ArrayList<>(documents)`	Oui	Oui
`Collections.unmodifiableList(documents)`	Non, un miroir	Non, mais suit l'original
`List.copyOf(documents)`	Oui, une photo	Non, refuse les null

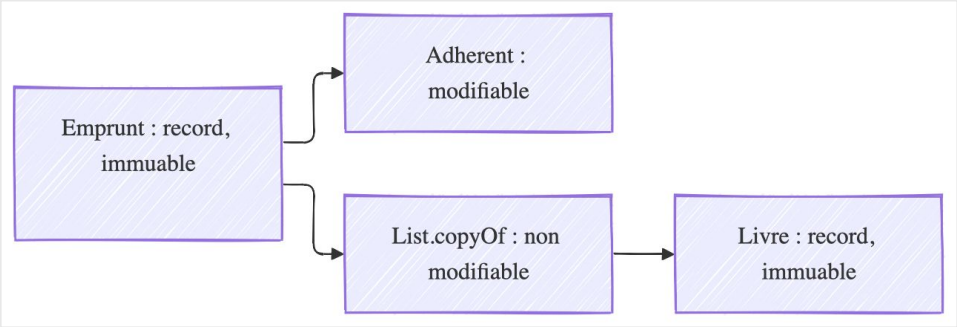
Je retiens : Pour une copie défensive, il faut les deux à la fois, une copie et la non-modifiabilité. Seul `List.copyOf` apporte les deux.

Immuable… en surface

- Un record garantit que ses composants désignent toujours les mêmes objets ⇒ la flèche ne bouge pas
- Si l'objet au bout de la flèche est modifiable ⇒ il peut changer
- L'immutabilité **profonde** exige des composants eux-mêmes immuables ⇒ Duree, Livre, une liste copiée de records

```
Emprunt emprunt = new Emprunt(alice, List.of(new Livre("Dune", 412)));

emprunt.adherent().emprunter();   // compile, et modifie Alice
```



Je retiens : Un record est immuable en surface. Un composant modifiable, comme une entité, peut toujours changer.

Hériter : toute la succession

Hériter, c'est accepter toute la succession : la maison, mais aussi les dettes.

- Une classe qui étend `ArrayList` hérite de ses 45 méthodes publiques ⇒ chacune est une porte d'entrée
- Redéfinir `add` ne ferme pas les autres portes ⇒ `addAll` et `add(int, …)` contournent la règle
- On dépend du fonctionnement interne d'ArrayList ⇒ un fonctionnement qu'on ne contrôle pas

Fichier Etagere.java

```
import java.util.ArrayList;

public class Etagere extends ArrayList<Document> {

    public static final int CAPACITE = 30;

    @Override

    public boolean add(Document document) {

        if (size() == CAPACITE) {

            throw new IllegalStateException("Étagère pleine");

        }

        return super.add(document);

    }

}
```

Hériter : toute la succession

```
Etagere etagere = new Etagere();
etagere.addAll(Collections.nCopies(40, new Livre("Dune", 412)));   // addAll n'appelle pas add
etagere.add(0, new Livre("Dune", 412));                            // add(int, …) non plus
System.out.println(etagere.size());   // 41 : la capacité de 30 est dépassée
```

Je retiens : L'héritage expose tout ce que la classe parente sait faire, y compris ce qui contourne vos règles.

La composition : avoir plutôt qu’être

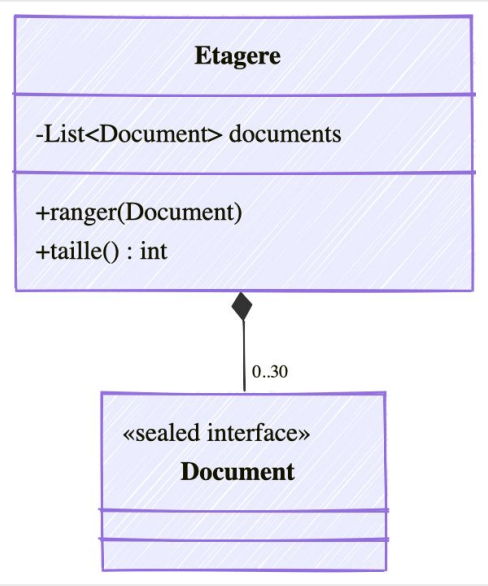
- Une étagère n’**est** pas une liste ⇒ elle **a** une liste, privée
- Elle n’expose que ce qu’elle choisit ⇒ ranger et taille
- La règle ne peut plus être contournée ⇒ il n’existe qu’une seule porte d’entrée

La composition : avoir plutôt qu'être

Fichier Etagere.java

```java
import java.util.ArrayList;
import java.util.List;
public final class Etagere {
    public static final int CAPACITE = 30;
    private final List<Document> documents = new ArrayList<>();
    public void ranger(Document document) {
        if (documents.size() == CAPACITE) {
            throw new IllegalStateException("Étagère pleine");
        }
        documents.add(document);
    }
    public int taille() {
        return documents.size();
    }
}
```

La composition : avoir plutôt qu'être



Je retiens :

- On n'hérite que si « est un » est vrai dans tous les cas, et si la classe parente est prévue pour être étendue. Sinon, on compose.
- Un record ne peut hériter de rien : avec les records, on compose naturellement. Un `Emprunt` a un `Adherent`.

Synthèse et travail à faire

Quel outil pour quel besoin ?

Je veux …	J'utilise …
Représenter une valeur (une durée, un prix)	Un record, validé dans son constructeur compact
Représenter quelque chose qui a une identité et qui évolue	Une classe `final`, avec `equals` et `hashCode` sur l'identifiant
Signaler une règle violée	Une exception non vérifiée, levée avant toute modification
Décrire une liste fermée de cas	Une interface `sealed` et des records
Traiter chaque cas	Un `switch` avec motifs, sans `default`
Recevoir une liste	`List.copyOf`
Réutiliser du code	La composition, plutôt que l'héritage

À vous

Récupérez les nouveautés : `git pull --no-edit prof main` (voir `README.md`, section « Routine d'un cours »).
Ouvrez ensuite `exercices/cours-02/enonce.md`.

TP

Le TP1 est publié dans **tp/tp1/enonce.md**. Il porte sur les cours 1 et 2.

Il est individuel et se fait hors séance

Les tests fournis sont dans **tp/tp1/tests/**

Le rendu est un zip, à envoyer avant le début de la séance 8 (voir README.md, section « Les TP »)

Attention

Respectez à la lettre les noms imposés, et l'en-tête de chaque fichier.

Les trois TP et le projet arrivent à échéance au même moment : avancez le TP1 sans attendre.

Projet … préparer la séance 3

Le cahier des charges du projet est publié dans *projet/cahier-des-charges.md*. Il donne les classes du socle et les méthodes de chaque partie.
En séance 3, chaque groupe présente la conception de ses parties et la répartition des rôles.

Lisez le cahier des charges avec les questions de ce cours.

- Quelles classes du socle sont des valeurs ? Un contrat a un numéro, et pourtant c'est un record. Pourquoi ?
- Pourquoi les zones et les garanties sont-elles des énumérations ?
- Pour votre partie, quelles classes allez-vous ajouter, et quelles règles doivent lever une exception ?

Apportez un diagramme de classes par partie.